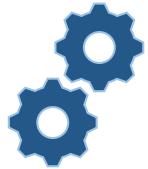


Why virtualization technologies?



Resource optimization
(maximize utilization)



Application isolation
(OS/libs compatibility)



Deployment flexibility
(scalability & migration)



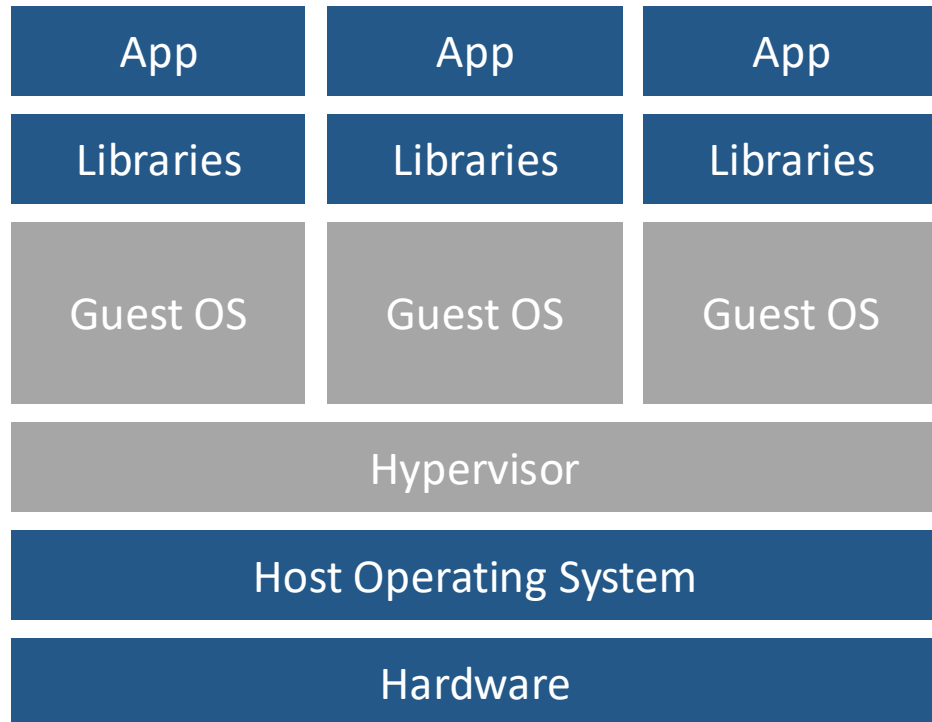
Server consolidation
(save administration effort)



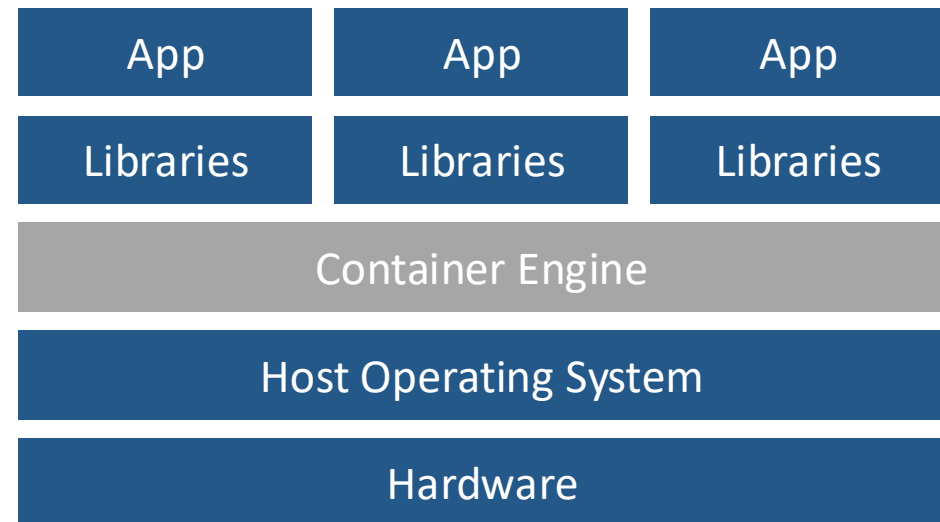
Testing and development
(simplify dev workflow)

Virtualization & Containerization

Virtual Machines



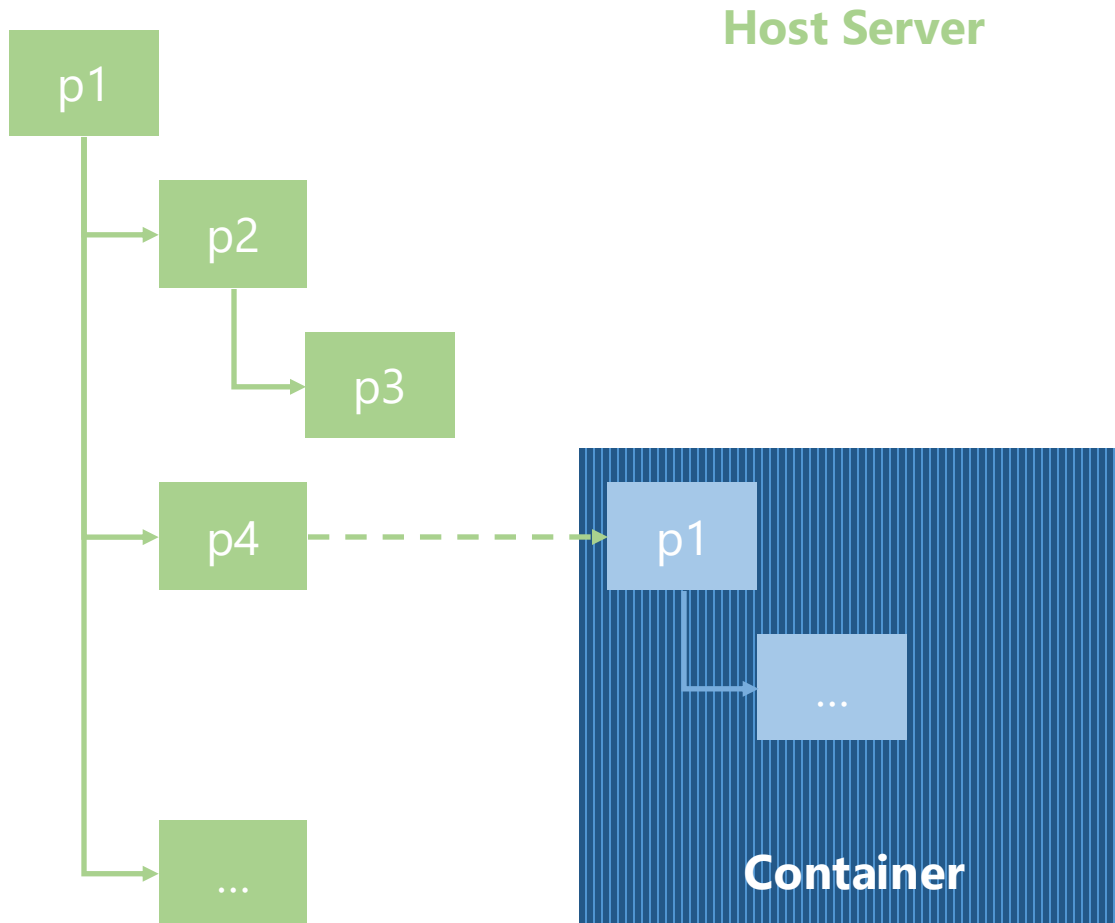
Containers



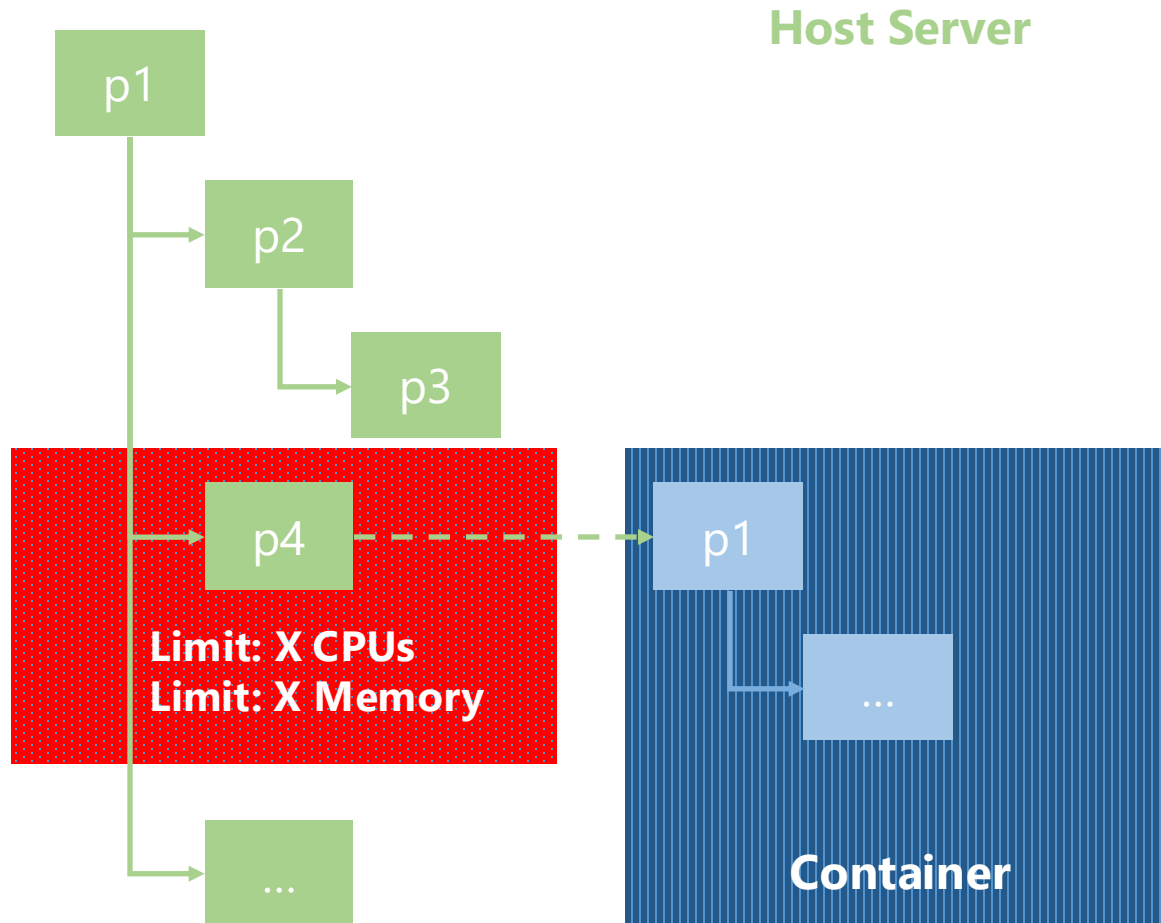
Virtualization & Containerization (Comparison)

Feature	Virtual machine	Container
Virtualization	Hardware level	OS level
Operating system	Has its own kernel	Shares the host operating system's kernel
Isolation	Fully isolated	Process-level isolation
Portability	Less portable	More portable
Speed	Slower to start up and shut down	Faster to start up and shut down
Resource usage	Uses more resources	Uses fewer resources
Use cases	Isolated applications	Portable and scalable applications

Containers (Namespaces)



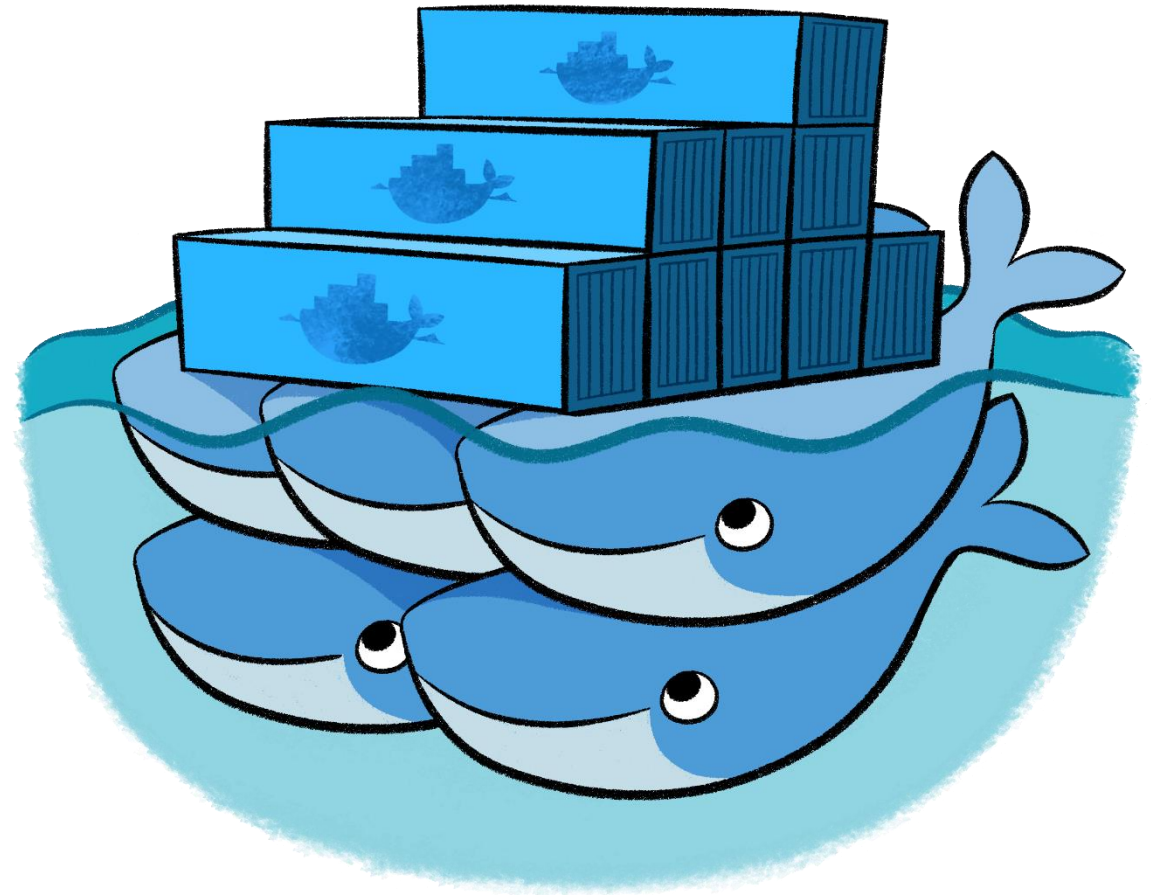
Containers (Control Groups)



Docker

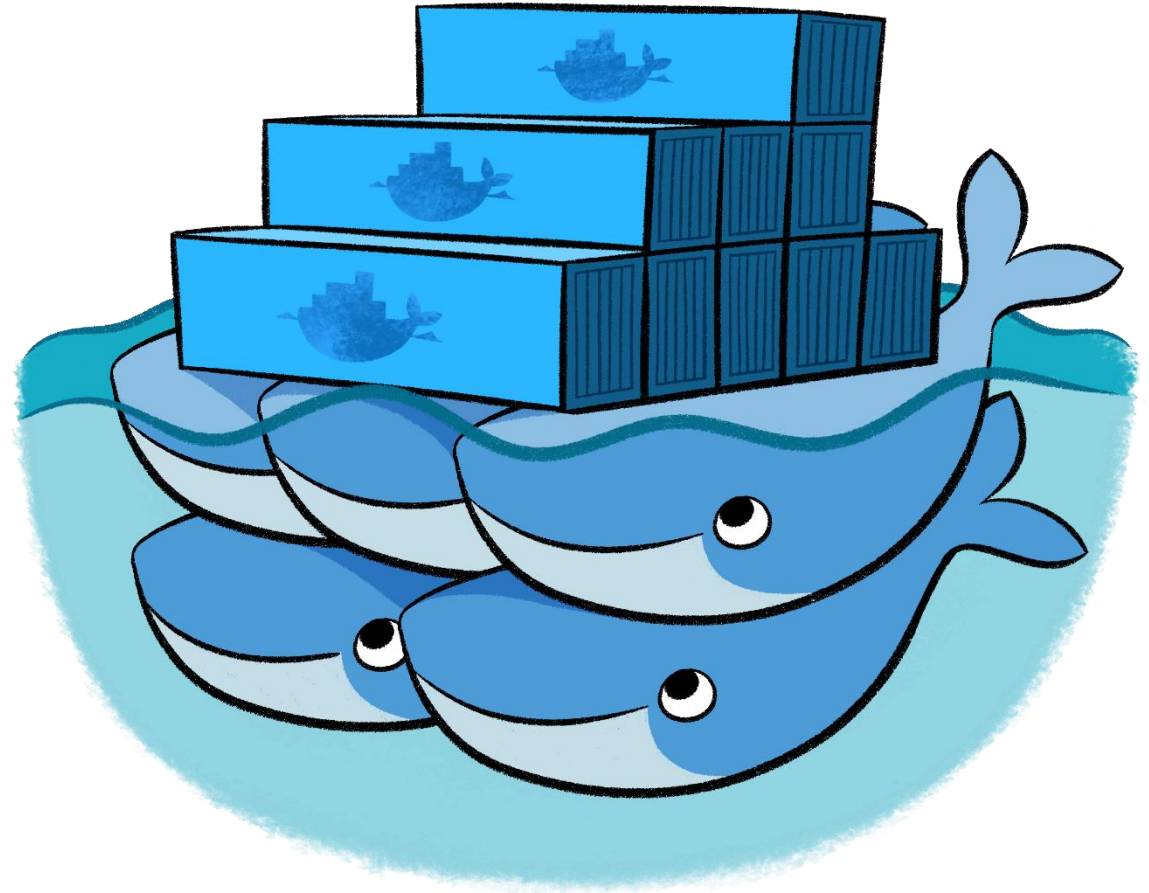
Leading open-source containerization platform

Docker containers wrap up a piece of software in a complete filesystem that contains everything it needs to run: code, runtime, system tools, system libraries – anything you can install on a server. This guarantees that it will always run the same, regardless of the environment it is running in

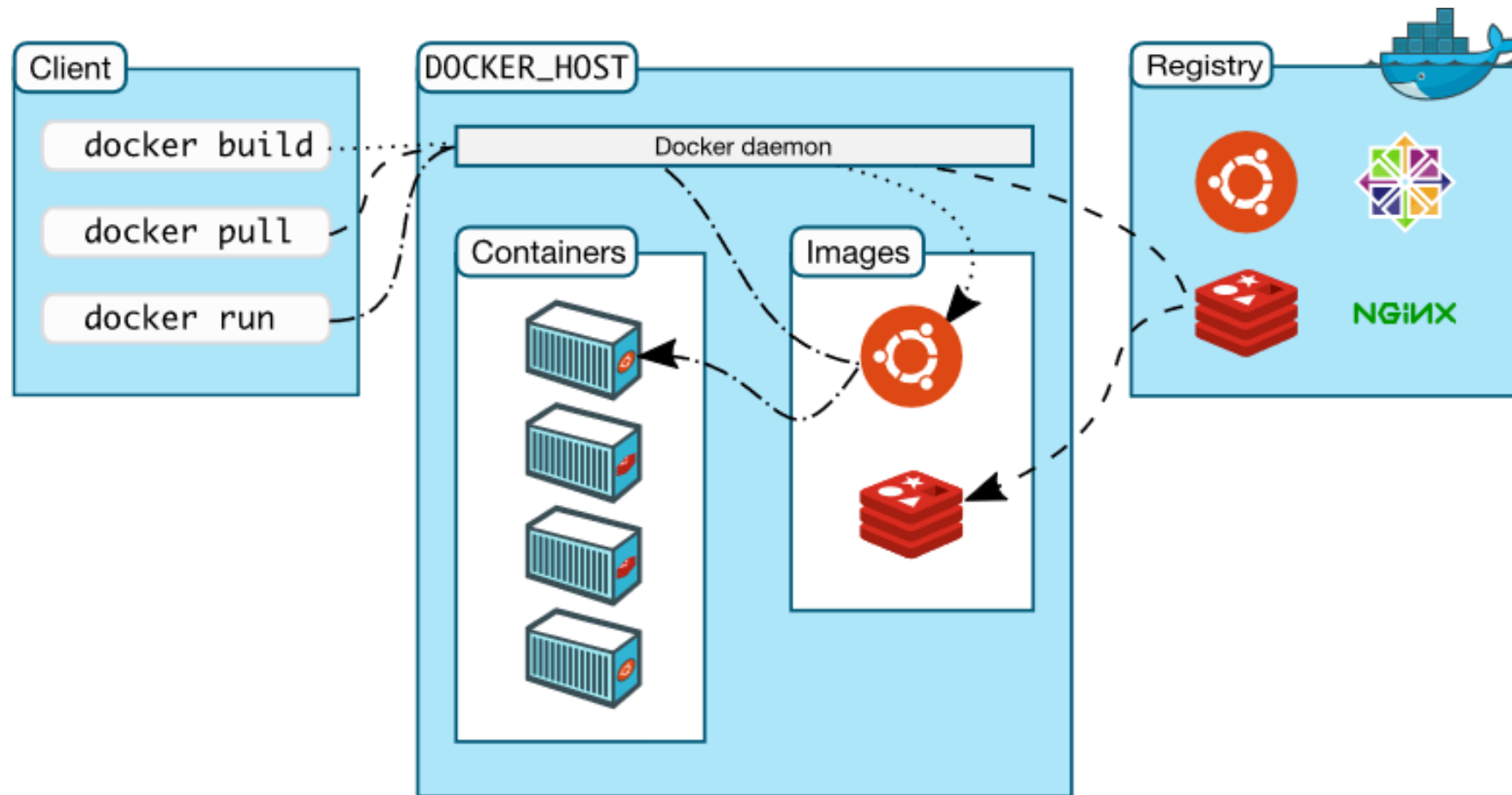


Docker (Installation)

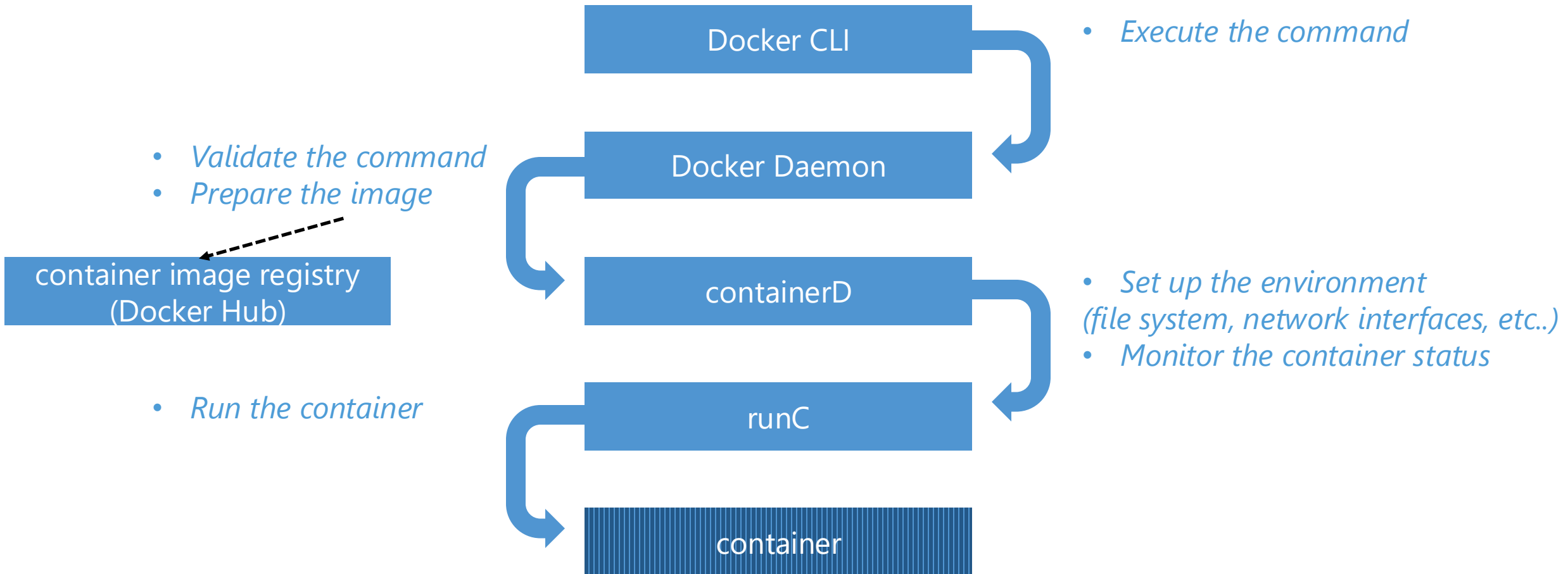
Select your supported platform
and follow the instructions to
[Install Docker Engine](#)



Docker (High-Level Architecture)



Docker (Flow)



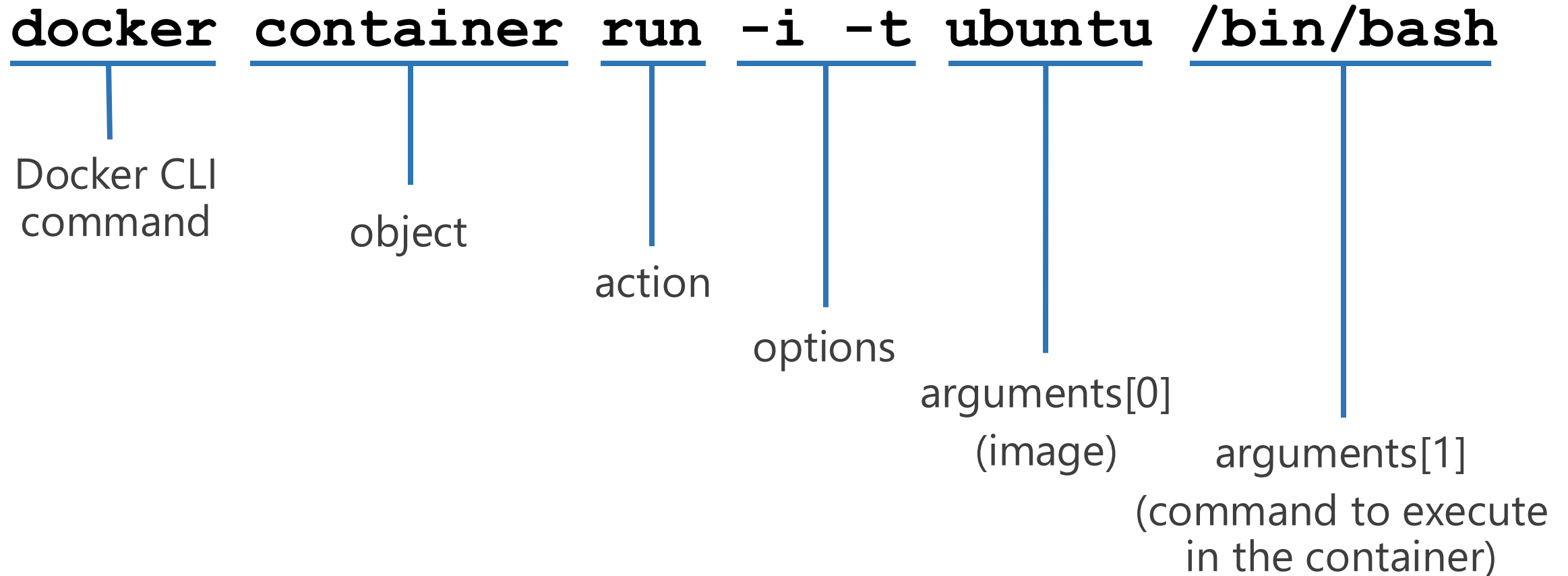
Docker CLI

- Command-line interface for Docker, available for Linux, OS X, and Windows (available separately or as part of Docker Toolbox)
- [docker container | Docker Docs](#)
- [docker image | Docker Docs](#)
- [docker_cheatsheet.pdf](#)



Docker CLI (Example)

Running a Container



Docker containers (Definition)

- A container is a standard unit of software that packages up code and all its dependencies, so the application runs quickly and reliably from one computing environment to another.
- It is the running instance of a template (Image).

Docker containers (Commands)

command	description
<code>docker container run</code>	Create and start a new container
<code>docker container start</code>	Start a container
<code>docker container stop</code>	Stop a container
<code>docker container rm</code>	Remove a container
<code>docker container exec</code>	Execute a command in a container
<code>docker container ls</code>	List containers
<code>docker container inspect</code>	Display detailed information of a container
<code>docker container logs</code>	Fetch the logs of a container
<code>docker container attach</code>	Attach local stdin, stdout, and stderr streams to a container
<code>docker container cp</code>	Copy files/folders between a container and the local filesystem

Container examples

run a container named my_container_1 in interactive mode from ubuntu image with an attached bash shell:

```
docker container run --name my_container_1 -it ubuntu bash
```

stop the container named my_container_1 :

```
docker container stop my_container_1
```

list all containers (running and stopped):

```
docker container ls -a
```

start the container named my_container_1 :

```
docker container start my_container_1
```

execute any command in the container named my_container_1 without interactive mode:

```
docker container exec my_container_1 cat /etc/os-release
```

list all information about the container named my_container_1:

```
docker container inspect my_container_1
```

remove the container named my_container_1:

```
docker container rm my_container_1
```

run a container named my_container_2 in detached mode from nginx image

```
docker container run --name my_container_2 -d nginx
```

show the logs of the container named my_container_2 :

```
docker container logs my_container_2
```

attach it back to your terminal:

```
docker container attach my_container_2
```

Docker containers (Demo)

- Play with container commands (ls/run/stop/start/rm/inspect/logs/...)
- Play with options:
 - --name <container-name>
 - -f (force)
 - -i (interactive)
 - -t (tty)
 - -d (detach)
 - --user <container-user>
- Additional options:
 - -e <KEY>=<VALUE> (Dockerfile section)
 - -p <host-port>:<container-port> (networks section)
 - -v </path/in/host/>:<path/in/container/> (storage section)

Docker containers (Notes)

- Gets a random name if not explicitly defined by the user.
- Can be referenced by its name or id in the commands.
- Keeps running as long as the main process (Entrypoint or CMD) is running.
- Should be stopped before removing it.
- Must be running to be able to “exec” on it.
- Can’t be mapped to an already used host port.
- Bind mounts & volumes data persist after the container is removed.
- The tag (latest) is used if not explicitly defined by the user.